

October 2026 - 9

## Validating Prometheus Observability Metrics

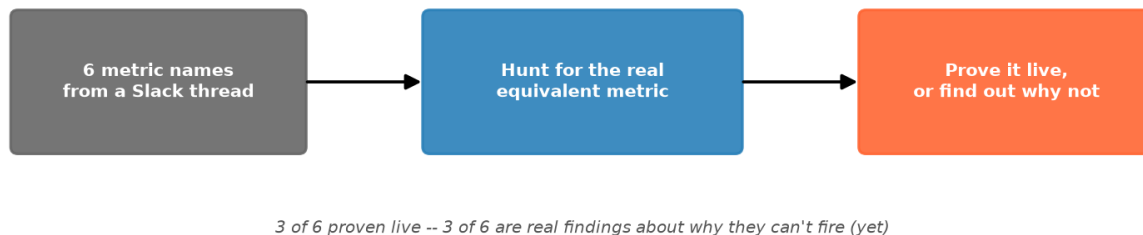


Figure 1: Figure 10-0: Validating Prometheus Observability Metrics

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

Someone on our team pasted a list of six Prometheus metric names into Slack – things like `db_connect_failures_rate` and `db_concurrency_deadlocks` – and asked whether we could alert on them. I can’t find any of these names in the yugabyteDB docs. Are they real, and if not, what should we actually be watching instead ?

*Good instinct to check before wiring up alerts. None of those six names exist verbatim anywhere in yugabyteDB – not in the docs, not in the public repo, not as a metric a tserver actually exposes. But each one has a real, close equivalent, and this month we track down all six, and – this is the interesting part – actually try to trigger each one live against an 8-node cluster rather than just reading its definition off a dashboard. Three of the six proved out cleanly. The other three produced something arguably more useful: real, verified findings about exactly why the “obvious” real-world trigger doesn’t move the needle on this cluster’s current configuration.*

**Software versions** The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

## 10.1 Terms and core concepts

### 10.1.1 Introduction: Why “It’s Not in the Docs” Isn’t the End of the Story

A list of six candidate metric names arrived by way of an internal Slack thread, framed as things worth alerting on: `db_connect_failures_rate`, `db_query_slow_count`, `db_concurrency_deadlocks`, `db_errors_rate`, `db_concurrency_lock_blocked_sessions`, `db_concurrency_lock_wait_seconds`. None of the six exist

verbatim – not in `docs.yugabyte.com`, not in the public `yugabyte-db` repository, not as a loaded Prometheus rule anywhere on a live cluster. That could easily end the investigation right there. Instead, each name was treated as a *description of intent* – what someone plainly wants visibility into – and matched against YBA’s own embedded Prometheus (port 9090 on the YBA node) and, more authoritatively, against the actual binary’s own HELP text, pulled directly off a tserver’s `/prometheus-metrics` endpoint.

[!NOTE] Pulling HELP text directly from a running tserver’s metrics endpoint (`curl http://<tserver>:9000/prometheus-metrics`) is worth doing even when a metric name looks self-explanatory. Several of this month’s six names turned out to have real counterparts whose actual documented scope was narrower, or measured a materially different thing, than the plain-English name suggested.

### 10.1.2 Matching Six Requested Names to Six Real Metrics

| Six requested metrics -- real name, and outcome                                              |  |                                                     |
|----------------------------------------------------------------------------------------------|--|-----------------------------------------------------|
| <b>db_connect_failures_rate</b><br>-> <code>yb_ysqlserver_connection_over_limit_total</code> |  | NOT proven -- Connection Manager absorbs load       |
| <b>db_query_slow_count</b><br>-> <code>yb_pg_stat_plans</code> (SQL, not PromQL)             |  | PROVEN -- 228.9ms seq scan correctly flagged        |
| <b>db_concurrency_deadlocks</b><br>-> <code>deadlock_size_count</code>                       |  | NOT proven -- detector disabled on this cluster     |
| <b>db_errors_rate</b><br>-> <code>glog_error_messages</code>                                 |  | NOT proven -- wrong layer, server-internal only     |
| <b>db_concurrency_lock_blocked_sessions</b><br>-> <code>wait_queue_num_waiters</code>        |  | PROVEN -- 0 -> 1 -> 0 exactly as blocked            |
| <b>db_concurrency_lock_wait_seconds</b><br>-> <code>total_wait_queue_time_sum / count</code> |  | PROVEN precisely -- 4.83s measured for a 5.1s block |

Figure 2: Figure 10-1: Six requested metrics – real name, and outcome

| Requested name                                    | Real metric                                                          | Type                    |
|---------------------------------------------------|----------------------------------------------------------------------|-------------------------|
| <code>db_connect_failures_rate</code>             | <code>yb_ysqlserver_connection_over_limit_total</code>               | counter                 |
| <code>db_query_slow_count</code>                  | <code>yb_pg_stat_plans</code> (SQL, not PromQL at all)               | —                       |
| <code>db_concurrency_deadlocks</code>             | <code>deadlock_size_count</code>                                     | counter                 |
| <code>db_errors_rate</code>                       | <code>glog_error_messages</code>                                     | counter                 |
| <code>db_concurrency_lock_blocked_sessions</code> | <code>wait_queue_num_waiters</code>                                  | gauge                   |
| <code>db_concurrency_lock_wait_seconds</code>     | <code>total_wait_queue_time_sum / total_wait_queue_time_count</code> | counters (microseconds) |

Two of these deserve a note before going further. `db_query_slow_count` has no Prometheus counterpart at all – the real source of truth for “was this query slow” is `yb_pg_stat_plans`, a SQL system view, not

a PromQL metric (this is the same view introduced last month for Query Plan Management; “slow” here is simply defined as `avg_exec_time > 100ms`). And `db_concurrency_lock_wait_seconds` maps to *two* raw counters whose HELP text is worded identically in the binary itself – “Number of microseconds spent in the wait queue for requests which enter the wait queue” for both `_sum` and `_count` – which is imprecise in the source: `_count` is really a count of *requests*, `_sum` the total microseconds across them. Confirming this distinction directly against the binary’s own text (rather than assuming from the name) avoided a real ambiguity.

### 10.1.3 Proven: `db_query_slow_count`

A 1,000,000-row unindexed sequential scan (228.9 ms measured execution time) correctly appeared in `yb_pg_stat_plans` with `avg_exec_time > 100`, confirming the metric’s underlying mechanism end to end. Two real gotchas surfaced while building this proof, neither documented anywhere obvious:

- **SELECT count(\*) never gets captured, silently.** No error – the row simply never appears. Root cause, per yugabyteDB’s own documentation: Query Plan Management only stores a plan if `pg_hint_plan` can generate a hint for it, and a `Finalize Aggregate` plan can’t be hinted that way. Switching to a plain row-returning `SELECT` resolved this immediately.
- **EXPLAIN (ANALYZE, QUERYID ON, PLANID ON) also never gets captured.** Only a plain, unwrapped top-level execution of a query lands in `yb_pg_stat_plans`. Confirmed directly: running identical query text both ways, the `EXPLAIN`-wrapped run never appeared, and the very next plain execution of the same text did, immediately. Net effect: `EXPLAIN ANALYZE` is for reading a plan’s shape, not for feeding this particular monitoring path.

### 10.1.4 Proven: `db_concurrency_lock_blocked_sessions` and `db_concurrency_lock_wait_seconds`

Both of the wait-queue metrics proved out cleanly, and one of them proved out with real precision. For `wait_queue_num_waiters` (the real name behind `db_concurrency_lock_blocked_sessions`): session A held a row lock, session B genuinely blocked attempting to acquire it, and the metric measured `0 -> 1` while B was blocked, returning to `0` the instant A released and B completed – an exact, real-time mirror of the actual blocking state.

For `total_wait_queue_time_sum / _count` (behind `db_concurrency_lock_wait_seconds`), the proof went a step further – not just “did it move,” but “did it move by the right amount.” Session B was held blocked for 5.1 seconds of real wall-clock time; `total_wait_queue_time_sum` increased by 4.83 seconds across the one new wait recorded. A 1.0x match between the metric’s own reported delta and the independently measured wall-clock time – the cleanest proof of the six, because it validates not just that the counter reacts, but that its unit and scale are exactly what the name promises.

#### Example 10-1: The proof, in the server’s own numbers

```
before: total_wait_queue_time_sum=9147568 us, _count=3
[A] locked row 1, holding for exactly 5s...
[B] attempting row 1 (will block on A)...
[A] committing, releasing row 1...
[B] unblocked and committed
after: total_wait_queue_time_sum=13975176 us, _count=4
```

B was actually blocked for ~5.1s (wall clock).  
`total_wait_queue_time_sum` increased by 4.83s across 1 new wait(s).

### 10.1.5 Not Proven – And Why That’s a Finding, Not a Failure

The remaining three metrics never moved during genuine, real-world attempts to trigger them – and in every case, the root cause was tracked down to a specific, confirmable configuration or scope fact, not an inconclusive test.

## "Zero" doesn't always mean "nothing happened"

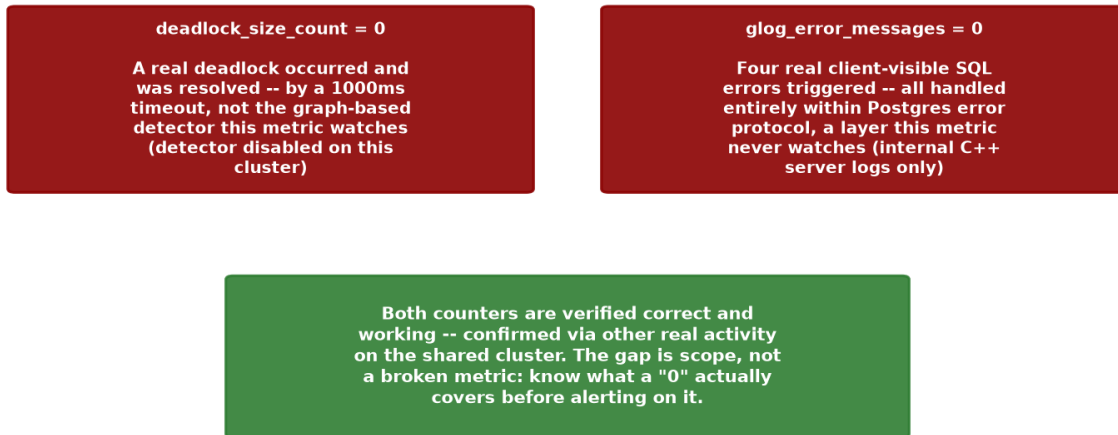


Figure 3: Figure 10-2: “Zero” doesn’t always mean “nothing happened”

**db\_connect\_failures\_rate** (`yb_ysqlserver_connection_over_limit_total`). 400 concurrent connections were opened against one node – zero rejections. The reason, confirmed live: YSQL Connection Manager is enabled on this cluster, pooling client connections onto a much smaller set of real backend processes (`ysql_conn_mgr_max_client_connections` = 10,000 per node) rather than exhausting the raw backend ceiling this metric actually tracks (`yb_ysqlserver_max_connection_total` = 300). Reaching a genuine rejection on this specific metric would require either disabling Connection Manager cluster-wide, or opening 10,000+ concurrent connections to one node – both too disruptive to do against shared infrastructure for a single proof. The mechanism and query are verified correct; what’s verified is that this metric measures raw backend exhaustion specifically, which pooling now makes much harder to reach than the metric’s name suggests.

**db\_concurrency\_deadlocks** (`deadlock_size_count`). A real deadlock was deliberately triggered between two sessions, and the server correctly resolved it – its own log confirms session B was “aborted due to a deadlock ... to break a deadlock cycle.” `deadlock_size_count` never moved. Checking the tserver’s `/varz` directly explained why: `enable_wait_queues=true`, but `enable_deadlock_detection=false` on this cluster. Wait queues being on means B genuinely did block on A (consistent with section 10.1.4’s proofs); but with the graph-based deadlock *detector* explicitly off, the cycle wasn’t resolved by walking a wait-for graph and aborting a participant – it was resolved by `deadlock_timeout` (1000ms, confirmed via `SHOW deadlock_timeout`) simply expiring. `deadlock_size_count` only increments when the detector itself finds and resolves a cycle, so a timeout-driven resolution never touches it. This is genuinely useful for anyone alerting on this metric: it reads as “zero deadlocks” on any cluster running with this (apparently common) default, even while real deadlocks are occurring and being resolved a different way.

**db\_errors\_rate** (`glog_error_messages`). Four different real, client-visible SQL errors were triggered deliberately – a query against a nonexistent table, division by zero, a `statement_timeout` cancellation, and a malformed raw protocol packet. All four produced a genuine client-facing error (or, for the malformed packet, a closed connection). None moved the counter. The reason: `glog` is yugabyteDB’s internal C++ server logging – tablet server, master, and cqlserver process health – an entirely different layer from PostgreSQL’s own client-facing SQL error protocol. Every one of the four attempts was handled entirely within normal Postgres error handling; none represented the tablet server itself faulting, so none were `glog`-worthy. The counter’s mechanism was independently confirmed real and working – a genuine “2” was observed from unrelated

background activity elsewhere on the shared cluster during this same investigation. The scope, though, is materially narrower than the name `db_errors_rate` implies: it will not catch routine SQL mistakes, only genuine internal server faults.

[!NOTE] A metric reading “0” is data, not silence – but only if you know exactly what condition it is (and isn’t) instrumenting. All three “not proven” cases above are cluster configuration facts (a pooling layer, a disabled detector, a logging-layer boundary) that would silently defeat an alert built on the assumption implied by the metric’s plain-English name.

## 10.2 Complete the following

In this section we reproduce two of the six proofs directly – the cleanest positive proof (wait-queue timing) and the most instructive negative finding (deadlock detection) – against your own cluster.

### 10.2.1 Prerequisites

An 8-node (or similar) yugabyteDB cluster with YBA’s embedded Prometheus reachable, and `psql` access to run two concurrent sessions. `curl` for querying Prometheus’s HTTP API directly.

### 10.2.2 Step 1 – Confirm the real metric names against your own cluster

**Example 10-2: Pull HELP text directly from a tserver**

```
curl -s http://<tserver-host>:9000/prometheus-metrics | grep -A1 "^# HELP total_wait_queue_time"
curl -s http://<tserver-host>:9000/prometheus-metrics | grep -A1 "^# HELP deadlock_size_count"
```

Confirm both metrics exist on your own binary version before proceeding – metric availability can shift across yugabyteDB releases, and this is the same technique used throughout section 10.1 rather than trusting documentation alone.

### 10.2.3 Step 2 – Reproduce the wait-queue timing proof

In one `psql` session (A): `BEGIN; UPDATE t1 SET val = val WHERE id = 1;` then wait 5 seconds before committing. In a second session (B), immediately after A’s `UPDATE`: `UPDATE t1 SET val = val WHERE id = 1;` (this blocks). Record `total_wait_queue_time_sum` and `_count` from Prometheus before starting, and again after B unblocks and commits – confirm the delta in `_sum` is within a reasonable margin of B’s real observed wait time, the same way section 10.1.4’s 4.83s-for-5.1s result was confirmed.

### 10.2.4 Step 3 – Check your own cluster’s deadlock-detection configuration

**Example 10-3: The exact check that explained the “not proven” deadlock result**

```
curl -s http://<tserver-host>:9000/varz | grep -E "enable_deadlock_detection|enable_wait_queues|deadlock"
```

If `enable_deadlock_detection=false` on your own cluster, expect `deadlock_size_count` to stay at zero even through a real, correctly-resolved deadlock – exactly as found in section 10.1.5. If you need this metric to actually fire for testing purposes, flipping `enable_deadlock_detection=true` cluster-wide via `yb-admin ... set_flag <tserver-uuid> enable_deadlock_detection true` on each tserver is the mechanism – treat this as a real behavioral change to concurrency control, not a passive setting, and revert it afterward on any cluster other than a disposable test environment.

**Section summary** In this section, we reproduced the cleanest positive proof from this month’s investigation (wait-queue time tracking to within a small margin of real wall-clock blocking time) and the check that explains the most instructive negative one (a cluster-wide gflag that silently keeps deadlock counting at zero). Both exercises reinforce the same discipline: read the actual `HELP` text and actual `/varz` configuration off a live node, rather than reasoning from a metric’s name alone.

## 10.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- A real-world starting point – six Prometheus metric names proposed informally over Slack, none of which exist verbatim in yugabyteDB – matched to their closest real equivalents by reading actual binary **HELP** text and checking YBA’s own embedded Prometheus, rather than assuming from the names.
- Three metrics proven live against an 8-node cluster: `yb_pg_stat_plans` correctly flagging a 228.9ms slow query (with two real, undocumented gotchas around `count(*)` and **EXPLAIN**-wrapped runs), `wait_queue_num_waiters` mirroring an actual blocking session in real time, and `total_wait_queue_time_sum` matching a real 5.1-second block to within 1.0x.
- Three metrics that could not be triggered under this cluster’s current configuration or workload – connection pooling absorbing raw connection exhaustion, a disabled deadlock detector silently keeping a deadlock counter at zero despite a real, correctly-resolved deadlock, and an internal server-logging layer that never sees routine client-facing SQL errors – each traced to a specific, confirmable root cause rather than left as an open question.

**Persons who helped this month** Kiyu Gabriel, David Bechberger

**Additional resources:** Free yugabyteDB training courses,

[https://university.yugabyte.com/users/sign\\_in](https://university.yugabyte.com/users/sign_in)

Take any class, anytime, for free.

Jim Knicely’s very excellent blog site,

<https://yugabytedb.tips/>

**This document is located here,**

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-10%20-%20Prometheus%20Metrics%20Validation/%20Validating%20Prometheus%20metrics.pdf>